Entangled Addresses: Recursive Zero-Knowledge Private Settlement for Post-Quantum Blockchains

Christopher Smith Ethan Cemer

March 2026

Abstract

Post-quantum blockchains face a basic tension: signature schemes that are believed to resist quantum attacks typically impose much larger public keys and signatures than classical elliptic-curve systems, while direct on-chain authorization also leaks transaction structure to every observer. We describe *Quantus Wormhole*, also referred to as *entangled addresses*, a private settlement mechanism that shifts authorization off-chain into recursive zero-knowledge proofs. A user deterministically derives an unspendable deposit address from a secret, funds that address through an ordinary transfer, and later proves in zero knowledge that (i) the funded record exists in authenticated chain state, (ii) the block-header commitment is valid, (iii) the corresponding nullifier is fresh, and (iv) the disclosed exits satisfy the protocol fee rule. The implementation uses Plonky2 recursion with Poseidon2-based hashing and a circuit split across nullifier, unspendable-account, storage-proof, dual-exit, and block-header fragments. The implemented first aggregation layer batches leaf proofs that share the same public `block_hash`; its wrapper forwards nullifiers, groups exit accounts, and hides batch occupancy with dummy padding and shuffling. This paper formalizes the leaf relation, separates proof-level and runtime-level security claims for deposit binding, one-time withdrawal, and value conservation, aligns the presentation with the implemented same-block aggregation model, and reports build, prove, and verification times for the refactored aggregation flow. On an Apple M2 Max running Rust 1.90.0, same-block first-layer batches of 2–49 leaves required 7.46–30.93 s to build, 3.72–32.42 s to prove, and 4.50–19.58 ms to verify; second-layer batches over 2–32 first-layer proofs with eight leaves each required 5.54–25.78 s to build, 3.52–21.32 s to prove, and 5.18–6.88 ms to verify.

1 Introduction

The migration from classical public-key cryptography to post-quantum alternatives is now a systems problem as much as a cryptographic one. Shor's algorithm invalidates the hardness assumptions behind the elliptic-curve signatures used by most deployed blockchains [12]. NIST's standardization of ML-DSA provides a practical post-quantum replacement path, but also foregrounds a cost already familiar to protocol engineers: public keys and signatures become substantially larger, pushing bandwidth, mempool pressure, state growth, and verification cost upward [7].

A naive response is to swap classical signatures for post-quantum ones and accept the resulting throughput loss. That approach is unsatisfactory for two reasons. First, it makes on-chain verification cost scale with the full signature representation of every spend. Second, it does nothing for privacy: the chain still exposes the exact account-level authorization graph of deposits, spends, and change outputs.

This paper studies an alternative. Instead of authorizing a spend directly on-chain, a user moves value into a deterministic address derived from a private secret and later authorizes release of that value with a zero-knowledge proof. The proof shows that an authenticated deposit exists in chain

state, that the claimant knows the secret corresponding to that deposit, that the claim has not been spent before, and that the revealed exits obey protocol fee constraints. Recursive composition then aggregates many such withdrawals into a much smaller number of on-chain verification events. The central design question is how to make this construction precise for a post-quantum setting and how to present the implemented protocol without mixing current behavior with older design notes.

The Quantus design builds on the wormhole concept proposed in EIP-7503 [6], but adapts it to the Quantus stack, to transparent recursion, and to a concrete storage-proof-based implementation. The result is not merely a conceptual privacy pool. It is a system with explicit circuit interfaces, a fixed leaf public-input layout, a concrete first-layer aggregation wrapper, and a lightweight verifier path in the surrounding codebase [11].

Contributions. We make the following contributions.

1. We formalize the *entangled-address* spending model for Quantus: deposits flow to deterministic unspendable addresses derived from a secret, and withdrawals are authorized by proving knowledge of that secret together with authenticated state inclusion.

2. We specify the implemented leaf relation in terms of public outputs, private witnesses, and sub-circuit constraints for nullifier derivation, unspendable-account derivation, storage-proof verification, fee-constrained dual exits, and block-header commitment.

3. We describe the implemented recursive aggregation architecture, emphasizing the normative behavior of the first aggregation layer: leaf proofs in a first-layer batch share the same public `block_hash`, are padded with dummy leaves when necessary, and are wrapped by a circuit that forwards nullifiers and grouped exits.

4. We translate the repository implementation into an academic-paper structure and report measured build, prove, and verify times for the refactored aggregation flow across first-layer batch sizes 2–49 and second-layer fan-ins 2–32.

5. We tighten the security argument by separating proof-level and runtime-level assumptions and by stating explicit claims for authenticated-deposit binding, first-layer wrapper soundness, one-time withdrawal per deposit record, spend-path exclusivity under the chain's address discipline, and global value conservation.

Scope note. Earlier proposal material described contiguous-block aggregation as a design direction [13]. This paper does *not* treat that behavior as implemented protocol logic. The normative design described here is the implemented same-`block_hash` first aggregation layer; older contiguous-block wording is retained only as historical context or future work.

2 Background and Design Goals

2.1 From post-quantum signatures to proof-based settlement

Post-quantum signature standards solve one problem while creating another: they preserve authenticity under stronger adversarial models, but they make direct authentication more expensive to publish and verify. In a blockchain, every byte of authorization data competes with user payload and consensus metadata, while every verification step competes with execution and state-transition work. This observation motivates a shift in architecture. Rather than publishing a large signature

for each spend, a user can prove knowledge of a short secret whose consequences are checked against authenticated chain state. The chain then verifies a proof and updates a nullifier set, rather than validating a large standalone signature for every transfer.

2.2 Relationship to wormholes and privacy pools

EIP-7503 proposes *zero-knowledge wormholes*: users deposit funds to addresses derived from secrets and later remint or unlock value by proving knowledge of those secrets in zero knowledge [6]. The Quantus design inherits the core idea that deposits to apparently ordinary-looking addresses can provide plausible deniability, but differs in three important ways. First, the system is designed explicitly with post-quantum-friendly proving in mind. Second, it is implemented against authenticated Quantus state rather than described only at the proposal level. Third, it includes a concrete recursive aggregation layer with dummy padding and grouped output forwarding.

The closest conceptual neighbors are therefore wormholes, privacy pools, and shielded systems such as Zcash and Tornado Cash [3, 5, 6, 10]. Quantus differs from fully shielded ledgers by proving against native chain state rather than introducing a separate note commitment tree as the primary state object.

2.3 Why transparent recursion

The implementation uses Plonky2, which combines a PLONK-style arithmetization with FRI-based commitments and supports recursive proof verification without a trusted setup [1, 9]. This makes it a natural fit for a system that wants both recursive aggregation and a post-quantum-oriented security story. Poseidon2 is used for in-circuit hashing because it is efficient over prime fields and avoids the bit-decomposition cost of conventional hash designs inside arithmetic circuits [4].

2.4 Design goals

The protocol aims to satisfy four simultaneous goals:

1. **Scalable authorization.** On-chain work should scale with proof verification and forwarded public outputs, not with raw post-quantum signature material per transaction.

2. **Authenticated settlement.** Every successful withdrawal must be bound to a real deposit recorded in authenticated chain state.

3. **Replay prevention.** A deposit should not authorize multiple withdrawals; nullifiers must uniquely bind proofs to deposits.

4. **Meaningful privacy.** The chain should reveal only the public information needed for settlement: exits, amounts, shared fee parameters, nullifiers, and a block commitment. Funding accounts, secrets, and raw storage proofs should remain private.

3 System, Adversary, and Trust Model

3.1 Roles

The system comprises the following roles.

- **User / wallet.** Generates a secret, derives an entangled address, funds it, collects the witness data for withdrawal, and may optionally perform first-layer aggregation locally.

- **Proof source or indexer.** Supplies the user with the relevant transfer record, block-header data, and storage proof needed to construct a witness. This role is not trusted for correctness because incorrect data causes proof failure.
- **First-layer aggregator.** A privacy-sensitive local actor that fills a batch with real and dummy leaf proofs, shuffles them, and produces the first recursive wrapper.
- **Higher-layer aggregator.** A potentially delegated actor that composes already aggregated proofs and forwards their public outputs.
- **Runtime / verifier.** Checks block consistency, fee parameters, nullifier freshness, and proof validity before applying settlement.

3.2 State objects

Let $\mathcal{A}$ be the set of 32-byte account identifiers and let σ denote chain state. A withdrawal witness references the following objects:

- an asset identifier a,
- an input amount v_{in}, quantized to the circuit's 0.01-unit resolution,
- a transfer counter c,
- a funding account α_{fund},
- an unspendable deposit account α_{wh},
- a processed storage proof authenticating the transfer record under a state root rt, and
- structured block-header fields ($\mathsf{parent_hash}, b, \mathsf{rt}, \mathsf{extrinsics_root}, \mathsf{digest}$).

The runtime additionally maintains a nullifier set $\mathcal{N}$ of all accepted nullifiers.

3.3 Adversarial capabilities

We assume a probabilistic polynomial-time adversary that can observe the full chain, submit arbitrary deposits, inspect public proof outputs, perform timing analysis, and attempt to replay or malform proof submissions. The adversary does *not* break the collision resistance of the hash function, the knowledge soundness of the proof system, or the chain's block-history availability assumptions.

3.4 Trust assumptions

The protocol relies on the following assumptions.

1. **Hash security.** The Poseidon2 instantiation used by the system remains collision resistant and preimage resistant for the relevant security level [4].
2. **Proof soundness.** The configured Plonky2 recursion stack remains knowledge sound under its standard assumptions [1, 9].
3. **Authenticated history.** The runtime can check that the public block commitment and block number correspond to a valid block in chain history.
4. **Witness availability.** Honest users can obtain the storage proof and block-header data needed to build a witness for their deposit.

Algorithm 1 High-level withdrawal pipeline

procedure WithdrawAndSettle$(s, c, \alpha_1, \alpha_2, v_1, v_2)$
  derive $\alpha_{\text{wh}} \leftarrow \mathsf{EA}(s)$
  fetch transfer record, block-header fields, and storage proof for counter c
  compute $n \leftarrow \mathsf{Null}(s, c)$
  form public inputs $x = (a, v_1, v_2, f, n, \alpha_1, \alpha_2, h_B, b)$
  form witness w from secret, transfer record, and authenticated state data
  produce leaf proof $\pi \leftarrow \mathsf{Prove}(x, w)$
  optionally aggregate π with same-block leaf proofs
  submit proof and public inputs for runtime verification
end procedure

4 Protocol Overview

The protocol consists of five stages.

1. Secret generation and deposit. The user samples a secret s and derives an entangled address

$$\mathsf{EA}(s) = H(H(\text{“}\mathsf{wormhole}'' \parallel s)).$$

The user then funds this address through an ordinary transfer on Quantus. The deposit is accompanied by a chain record that can later be authenticated inside the circuit.

2. Witness acquisition. To withdraw, the user gathers the transfer record, a processed storage proof, the state root to which that proof authenticates, and the structured block-header fields needed to recompute the public block commitment. These data form the private witness except for the public block fields that will be exposed by the proof.

3. Leaf proof generation. The user chooses up to two exit outputs, computes a nullifier

$$\mathsf{Null}(s, c) = H(H(\text{“}\tilde{}\mathsf{nullif}\tilde{}'' \parallel s \parallel c)),$$

and produces a leaf proof. The proof reveals the nullifier, the exits, the output amounts, the asset identifier, the fee basis points, the public block commitment, and the block number, while keeping the secret, funding account, storage proof, and transfer-specific raw details private.

4. First-layer aggregation. A privacy-sensitive local aggregator batches leaf proofs that share the same public `block_hash`. If the batch is not full, the aggregator pads it with dummy leaves, shuffles the order, recursively merges the proofs, and then applies a wrapper circuit that enforces consistency and forwards grouped exits and nullifiers.

5. Settlement. The runtime verifies the aggregate proof, checks that the public `block_hash` matches the referenced block number, verifies that the fee parameter matches runtime configuration, rejects reused nullifiers, and settles the exits.

Field	Type	Meaning
`asset_id`	`u32`	Token identifier; `0` denotes the native asset.
`output_amount_1`	`u32`	Primary output amount after quantization.
`output_amount_2`	`u32`	Secondary output amount; zero if unused.
`volume_fee_bps`	`u32`	Runtime fee parameter in basis points.
`nullifier`	4 felts	Public replay-prevention tag derived from secret and transfer counter.
`exit_account_1`	4 felts	Primary destination account.
`exit_account_2`	4 felts	Secondary destination account or all zeros.
`block_hash`	4 felts	Public Poseidon2 commitment to structured block-header fields.
`block_number`	`u32`	Height of the block whose header is committed by `block_hash`.

Table 1: Logical leaf public inputs. In the current implementation this logical interface occupies 21 field elements.

Field	Type	Meaning
`secret`	32 bytes	User secret from which both nullifier and entangled address are derived.
`transfer_count`	`u64`	Per-transfer counter used in nullifier derivation.
`funding_account`	4 felts	Source account associated with the transfer record.
`unspendable_account`	4 felts	Derived entangled address $\mathsf{EA}(s)$.
`storage_proof`	structured witness	Processed inclusion proof for the transfer record under the state root.
`state_root`	4 felts	Root authenticating the storage proof.
`parent_hash`	4 felts	Parent block hash used in header recomputation.
`extrinsics_root`	4 felts	Extrinsics root from the block header.
`digest`	fixed bytes	Encoded digest logs.
`input_amount`	`u32`-compatible quantity	Quantized deposited amount before fee deduction.

Table 2: Private witness components used by the leaf proof.

5 Formal Specification and Interfaces

5.1 Leaf public inputs

The implemented leaf proof exposes exactly nine logical public fields. In the repository layout, these occupy 21 field elements because the nullifier, each exit account, and the block hash each consume four field elements [11].

5.2 Leaf private witness

The witness consists of the user's secret, the transfer counter, the funding account, the derived unspendable account, the processed storage proof, the state root, the parent hash, the extrinsics root, the encoded digest logs, and the input amount recorded in storage.

5.3 Leaf relation

Let $x = (a, v_1, v_2, f, n, \alpha_1, \alpha_2, h_B, b)$ denote the public input tuple and let w denote the witness described above. The leaf relation $\mathcal{R}_{\text{leaf}}$ holds when there exists a witness w such that:

(C1) **Nullifier correctness.** $n = \mathsf{Null}(s, c)$.

(C2) **Entangled-address correctness.** $\alpha_{\text{wh}} = \mathsf{EA}(s)$.

(C3) **Authenticated inclusion.** The processed storage proof verifies that a transfer record keyed by the relevant funding account, entangled account, and counter exists under rt.

(C4) **Header binding.** The structured block-header fields hash to the public commitment h_B, and the public block number equals the header's block number.

(C5) **Fee constraint.** The disclosed exits satisfy

$$(v_1 + v_2) \cdot 10000 \leq v_{\text{in}} \cdot (10000 - f).$$

(C6) **Range constraints.** The public quantities lie within the ranges expected by the runtime interface.

Definition 1 (Leaf relation)**.** *The relation $\mathcal{R}_{\text{leaf}}$ is the set of all (x, w) satisfying conditions (C1)–(C6).*

5.4 Runtime interface

The proof system does not, by itself, settle a withdrawal. The runtime must also check the following: (i) the referenced block exists in chain history, (ii) the public fee parameter matches the active runtime fee configuration, (iii) the public nullifier has not appeared before, and (iv) any asset-specific accounting rules are satisfied. These external checks are part of the overall security argument.

6 Leaf Circuit Design

The implemented leaf circuit is assembled from specialized fragments [11]. This modular structure cleanly separates correctness arguments.

6.1 Nullifier circuit

The nullifier fragment allocates the public nullifier and proves consistency with the secret and transfer counter. In the current implementation, the equality constraint to the public hash is gated so that dummy leaves can bypass the check while preserving overall wrapper validity.

6.2 Unspendable-account circuit

The unspendable-account fragment proves that the deposit destination equals the double-hash derivation of the secret under the protocol tag `"wormhole"`. This binds a user's secret to the on-chain deposit address without revealing the secret itself.

6.3 Storage-proof circuit

The storage-proof fragment is the most substantial component. It verifies a processed Merkle-style proof that a transfer record exists under the provided state root and exposes the fields needed by the fee and accounting logic. This is the bridge between off-chain proving and authenticated chain state.

6.4 Dual-exit circuit

The dual-exit fragment models a two-output withdrawal with Bitcoin-like "spend plus change" semantics. We refer to the current interface as a *two-output model* because that term accurately captures the interface and highlights a practical limitation: the protocol does not yet expose an arbitrary-length output vector.

6.5 Block-header circuit

Instead of taking an opaque serialized header as a monolithic input, the circuit receives structured header components and recomputes a public commitment from them. This improves specification clarity and makes it easier to state exactly what the proof binds to.

6.6 Dummy-leaf semantics

Dummy leaves are used to pad partially filled batches. The current implementation identifies a dummy by the sentinel condition `block_hash = 0^32`. Constraints tied to real deposits, such as nullifier consistency or block-header correctness, are conditionally disabled for dummies; however, the wrapper later replaces dummy nullifiers with fresh random values so that dummy leaves cannot collide on-chain. Dummy padding is therefore not merely an engineering convenience; it is part of the privacy story because it obscures batch occupancy.

7 Recursive Aggregation Architecture

7.1 Leaf layout and wrapper output layout

Each leaf proof contributes 21 public-input field elements. The first aggregation layer produces a wrapper proof whose semantic public output consists of:

- a fixed 8-felt header containing the number of exit slots, asset identifier, fee basis points, block hash, and block number;
- $2N$ exit slots, each represented as a 1-felt amount plus a 4-felt exit account; and
- N nullifiers, each occupying 4 felts.

Thus the meaningful forwarded payload is $14N + 8$ felts, although the current wrapper pads its public-input vector out to $21N + 8$ felts for layout regularity.

7.2 Normative first-layer policy

The first aggregation layer is the privacy-sensitive layer and the only aggregation layer for which this paper makes normative implementation claims. A valid first-layer batch must satisfy the following policy.

1. Every real leaf proof in the batch shares the same public `block_hash`.
2. Every real leaf proof shares the same `asset_id` and `volume_fee_bps`.
3. If fewer than N real leaves are available, the batch is padded with dummy leaves.
4. The leaf order is shuffled before wrapping, subject to implementation-specific handling that preserves at least one real leaf in a stable position for correctness.

5. Duplicate exit accounts are grouped so that identical destinations accumulate a summed amount in one slot while duplicate slots are zeroed.

6. All nullifiers are forwarded, with dummy nullifiers replaced by fresh random values.

We state this same-block rule explicitly to avoid ambiguity with the older contiguous-block wording in earlier proposal text.

7.3 Wrapper logic

At a high level, the wrapper circuit performs four tasks.

1. It detects dummy leaves by checking whether the child `block_hash` is all zeros.

2. It enforces that every non-dummy child agrees with the reference child's `block_hash`, `asset_id`, and `volume_fee_bps`.

3. It computes grouped exit slots by summing amounts for identical exit accounts and zeroing duplicate slots.

4. It forwards real nullifiers and substitutes random dummy nullifiers for dummy children.

7.4 Higher-layer aggregation

Higher recursive layers operate over already aggregated proofs rather than raw leaf witnesses. This separation is important: the privacy-critical shuffle and dummy-handling logic stays local to the first layer, while later layers can

8 Security and Privacy Analysis

This section separates *proof-level* statements from *system-level* statements. Proof-level statements concern the relations enforced by the leaf circuit and the first-layer wrapper. System-level statements additionally rely on runtime checks outside the proof system: the public `block_hash`/`block_number` pair must refer to a real block in chain history, the public fee parameter must match runtime configuration, nullifiers must be rejected on reuse (including duplicates inside one candidate batch), and settlement must apply exactly the forwarded exit amounts. Appendix A expands the proof sketches below into code-aware reductions tied to the current dummy-template and artifact-loading paths.

8.1 Authenticated deposit records

To tighten the security argument, it is useful to name the object that the proof is actually certifying.

Definition 2 (Authenticated deposit record)**.** *An* authenticated deposit record *is a tuple*

$$d = (a, v_{\text{in}}, c, \alpha_{\text{fund}}, \alpha_{\text{wh}}, h_B, b)$$

for which there exists witness data w such that $(x, w) \in \mathcal{R}_{\text{leaf}}$, where x exposes asset identifier a, block commitment h_B, block number b, and exits satisfying the fee constraint against v_{in}. Intuitively, d is a deposit object that has been authenticated by the storage-proof and header-binding subcircuits described in sections 5 and 6.

A *real* leaf is a leaf whose public `block_hash` is nonzero. A *dummy* leaf is a padding object identified by the sentinel `block_hash` $= 0^{32}$ and used only inside first-layer aggregation.

Claim	Additional assumptions outside the circuit relation
Leaf-to-deposit binding	Runtime validates that the public `block_hash`/`block_number` pair denotes a real block and that the processed storage proof format matches the authenticated state representation.
First-layer wrapper soundness	The recursive verifier gadgets correctly enforce child-proof validity, and the wrapper implementation matches its stated grouping and forwarding logic.
One-time withdrawal per authenticated deposit record	Runtime rejects reused nullifiers, including duplicates within one aggregate submission; the authenticated transfer record fixes the transfer counter used in nullifier derivation.
Spend-path exclusivity for entangled addresses	The surrounding chain only admits direct spends from addresses backed by valid external credentials, and no efficient adversary can manufacture such credentials for an entangled address.
Global value conservation	Runtime settles exactly the forwarded public outputs and records the associated nullifiers.

Table 3: Where the security argument relies on runtime or chain-interface assumptions beyond circuit correctness.

8.2 Completeness

Theorem 1 (Leaf completeness)**.** *If an honest prover holds witness data w for which $(x, w) \in \mathcal{R}_{\text{leaf}}$, then the prover can generate a leaf proof accepted by the leaf verifier.*

Proof sketch. Completeness follows from the correctness of the individual circuit fragments and of the proving system. Each fragment enforces a deterministic relation between its witness variables and public outputs. When the nullifier, entangled address, storage proof, header commitment, fee inequality, and range constraints are all satisfied, the assembled witness satisfies $\mathcal{R}_{\text{leaf}}$, so the prover can produce an accepting proof. □

Theorem 2 (Wrapper completeness)**.** *Suppose a first-layer aggregator is given a full batch of valid child proofs, padding any missing positions with the designated dummy template. If all real children share the same public* `block_hash`*,* `asset_id`*, and* `volume_fee_bps`*, then the aggregator can produce an accepting first-layer wrapper proof.*

Proof sketch. The wrapper verifies the child proofs recursively, checks equality of the shared batch parameters across all non-dummy children, applies the implemented exit-grouping logic, and forwards nullifiers together with dummy replacements. When the inputs satisfy those conditions, the wrapper constraints are satisfiable by construction. □

8.3 Leaf soundness and deposit binding

Theorem 3 (Leaf soundness)**.** *Assume the leaf proof system is knowledge sound. For every accepted leaf proof (x, π), there exists an extractor that outputs a witness w such that $(x, w) \in \mathcal{R}_{\text{leaf}}$.*

Proof sketch. This is the standard consequence of knowledge soundness for the leaf relation. The extractor outputs values for the secret, counter, funding account, entangled account, storage-proof witness, header fields, and input amount that satisfy conditions (C1)–(C6) from section 5. No

further claim is made here about whether the committed block actually exists in canonical chain history; that is deferred to the runtime check on the public `block_hash`/`block_number` pair. □

Corollary 4 (Authenticated-deposit binding)**.** *Assume leaf soundness and correct runtime validation of the public block commitment. Then every accepted real leaf proof binds to an authenticated deposit record and certifies that its disclosed exits satisfy the protocol fee rule for that record.*

Proof. By leaf soundness, an accepting real leaf yields a witness satisfying $\mathcal{R}_{\text{leaf}}$. Conditions (C3) and (C4) bind the proof to an authenticated transfer record under the state root committed by the structured header fields, while condition (C5) enforces the fee inequality. Runtime validation upgrades the public header commitment from a merely self-consistent header witness to a real block in chain history. □

8.4 First-layer wrapper soundness

Theorem 5 (First-layer wrapper soundness)**.** *Assume the recursive verifier gadgets and the wrapper circuit are sound. Let Y be the public output of an accepted first-layer wrapper proof of arity N. Then there exists a list of child instances*

$$((x_1, \pi_1, \delta_1), \ldots, (x_N, \pi_N, \delta_N)),$$

where $\delta_i \in \{0, 1\}$ indicates whether child i is dummy, such that:

(W1) if $\delta_i = 0$, then π_i is an accepted leaf proof on public input x_i;

(W2) if $\delta_i = 1$, then child i is recognized by the dummy sentinel `block_hash` $= 0^{32}$;

(W3) all non-dummy children share the same public header tuple exposed by Y: a common block hash, asset identifier, fee parameter, and block number;

(W4) the exit slots exposed by Y are exactly the result of applying the wrapper's grouping rule to the multiset of non-dummy child exits; and

(W5) the nullifier region of Y contains every real child nullifier, while dummy positions are filled by wrapper-supplied replacement values.

Proof sketch. Soundness of the recursive verifier gadgets ensures that every accepted child proof referenced by the wrapper witness is itself valid. The wrapper then recomputes its public outputs from those children subject to explicit conditional checks: non-dummy children must agree on the batch `block_hash`, `asset_id`, and `volume_fee_bps`; exit slots are formed by the implemented grouping-and-zeroing logic; and nullifiers are forwarded child-by-child with dummy substitution in positions marked by the sentinel. The shared `block_number` claim is obtained by combining wrapper soundness with leaf soundness: each real child is itself a sound leaf proof satisfying condition (C4), so a common `block_hash` implies a common committed header and hence a common `block_number`. A line-by-line version of this argument, together with the exact code-path linkage for dummy padding, is given in appendix A. Hence any accepting wrapper proof can be explained by a concrete list of real and dummy children satisfying (W1)–(W5). □

Lemma 6 (Dummy-padding safety)**.** *Assume the designated dummy template exposes zero-valued exits, and the runtime only settles the public exits produced by the wrapper. Then dummy padding cannot increase settled value. Random replacement of dummy nullifiers affects only accidental-collision probability and therefore liveness, not soundness.*

Proof. Under the stated dummy-template assumption, a dummy child contributes no positive exit amount to the grouped wrapper outputs. Replacing its nullifier with a fresh filler does not create an authenticated deposit record or any additional settled value; at worst, an accidental nullifier collision causes rejection of the aggregate. Thus dummy padding cannot authorize value that was absent from the real children. □

8.5 One-time withdrawal and double-spend resistance

Theorem 7 (One-time withdrawal per authenticated deposit record)**.** *Assume:*

(D1) the runtime rejects reused nullifiers, including duplicates within one candidate aggregate submission;

(D2) the authenticated transfer record fixes the transfer counter c used in nullifier derivation; and

(D3) the hash function used in $\mathsf{EA}(\cdot)$ and $\mathsf{Null}(\cdot,\cdot)$ is collision resistant and second-preimage resistant for the relevant domain-separated inputs.

Then no probabilistic polynomial-time adversary can obtain two accepted real withdrawals from the same authenticated deposit record except with negligible probability.

Proof sketch. Suppose two accepted real withdrawals correspond to the same authenticated deposit record. By authenticated-deposit binding, extraction yields witnesses (s, c) and (s', c') together with the same authenticated record

$$(a, v_{\mathrm{in}}, c, \alpha_{\mathrm{fund}}, \alpha_{\mathrm{wh}}, h_B, b).$$

Because the record fixes the counter, we have $c = c'$. Because the record also fixes the entangled address, both witnesses satisfy

$$\mathsf{EA}(s) = \alpha_{\mathrm{wh}} = \mathsf{EA}(s').$$

If $s \neq s'$, the adversary has found a collision or second preimage for the entangled-address derivation. Therefore $s = s'$ except with negligible probability. But then

$$\mathsf{Null}(s, c) = \mathsf{Null}(s', c'),$$

so both withdrawals expose the same nullifier. The runtime rejects the second occurrence by assumption (D1). Hence one authenticated deposit record can underwrite at most one accepted real withdrawal. □

The theorem above is intentionally phrased in terms of an *authenticated deposit record*, not an informal notion of "the same funds." If a future protocol version supports explicit partial-withdrawal state transitions, that stronger state model should be proved separately rather than inferred from the present one-shot nullifier discipline.

8.6 Operational unspendability and spend-path exclusivity

The protocol needs one assumption that is partly external to the circuit: funds sent to an entangled address should not also be directly spendable through an ordinary account-authentication path.

Proposition 8 (Spend-path exclusivity under the chain's address discipline)**.** *Let $\mathsf{Addr} : \mathcal{PK}_\Sigma \to \{0,1\}^{256}$ denote the chain's ordinary address derivation from externally controlled public keys. Suppose:*

(U1) every direct spend accepted by the base chain must be authorized by a valid public key $pk \in \mathcal{PK}_\Sigma$ whose address is $\mathsf{Addr}(pk)$; and

(U2) no efficient adversary can find $pk \in \mathcal{PK}_\Sigma$ such that $\mathsf{Addr}(pk) = \mathsf{EA}(s)$ for a target entangled address, except by breaking the collision or preimage resistance of the derivation.

Then value sent to $\mathsf{EA}(s)$ can leave the system only through the proof-authorized settlement path.

Proof sketch. Any direct spend must use some valid external credential pk by assumption (U1). If such a spend exists from $\mathsf{EA}(s)$, then $\mathsf{Addr}(pk) = \mathsf{EA}(s)$, contradicting (U2) except with negligible probability. Therefore the only remaining spend path is the proof-authorized one. We treat spend-path exclusivity as a system assumption about the surrounding chain interface rather than as a theorem of the circuit in isolation. □

8.7 Value conservation

Lemma 9 (Per-leaf fee bound)**.** *For every accepted real leaf proof, the sum of its two public output amounts is at most the authenticated input amount net of the public fee parameter.*

Proof. This is exactly condition (C5) of $\mathcal{R}_{\text{leaf}}$, obtained by extraction from leaf soundness. □

Lemma 10 (Wrapper amount preservation)**.** *Assume the first-layer wrapper groups identical exit accounts by summing their amounts into one slot and zeroing duplicate slots. Then the sum of the forwarded exit amounts equals the sum of the exit amounts of the real child proofs.*

Proof. The wrapper's grouping step is a partition of the multiset of real child exits by destination account. Replacing each partition class by a single slot whose amount is the class sum preserves the total amount. Zeroing duplicate slots removes representation duplicates, not value. □

Theorem 11 (Global value conservation)**.** *Assume:*

(V1) leaf soundness,

(V2) first-layer wrapper soundness,

(V3) runtime rejection of reused nullifiers, and

(V4) runtime settlement that applies exactly the forwarded public exit amounts.

Then the total value settled by any accepted first-layer aggregate proof is at most the total value of its underlying authenticated deposit records after protocol fees. Across multiple accepted proofs, no authenticated deposit record is counted more than once.

Proof sketch. By first-layer wrapper soundness, an accepted aggregate expands to a multiset of real child leaves and dummy padding. By dummy-padding safety, dummies contribute no settled value. By authenticated-deposit binding and the per-leaf fee bound, each real child contributes at most its authenticated deposit amount net of fees. By wrapper amount preservation, the sum of forwarded exit amounts equals the sum of the real child exit amounts. Finally, one-time withdrawal per authenticated deposit record rules out counting the same authenticated record twice via multiple accepted nullifiers. Hence settled value never exceeds the aggregate authenticated input value net of fees. □

Crate	Role
`qp-wormhole-inputs`	Shared public-input and aggregated-output data structures.
`qp-wormhole-circuit`	Leaf circuit definition, witness targets, and circuit fragments.
`qp-wormhole-prover`	Prover API for witness commitment and proof generation.
`qp-wormhole-verifier`	Lightweight verifier-facing interface.
`qp-wormhole-aggregator`	Recursive aggregation and wrapper logic.
`qp-wormhole-circuit-builder`	Serialization of prebuilt circuit artifacts for deployment.

Table 4: Repository-level implementation split.

8.8 Privacy boundary and leakage model

The security arguments above are intentionally narrower than a full anonymity theorem. There are two distinct privacy questions.

First, there is *proof-level witness hiding*: secrets, funding accounts, storage-path details, and raw header components are not exposed as public inputs, so any leakage about them must come from the proof system itself or from side channels. This paper treats that as a property inherited from the concrete proving-system configuration rather than as a standalone contribution.

Second, there is *system-level transaction privacy*. Here the public interface is deliberately revealing: output amounts, exit accounts, fee basis points, nullifiers, `block_hash`, and `block_number` are all public. Dummy padding helps hide batch occupancy, and exit grouping weakens direct child-to-output correspondence, but amount leakage, address reuse, timing correlation, and small anonymity sets remain important limitations. Accordingly, the security claim is limited to hiding witness fields that are not deliberately re-exposed; the protocol does *not* provide a full simulation-based unlinkability guarantee over all candidate deposits.

9 Implementation

The current codebase is organized as a family of crates rather than as a monolithic proving binary [11]. That modularization is worth surfacing explicitly in the paper because it clarifies where proof construction, verification, and aggregation responsibilities live.

9.1 Commit-then-prove API

The prover follows a two-step interface: a `commit` stage fills the witness, and a subsequent `prove` stage runs the expensive proving work. This separation is useful both for engineering and for benchmarking because it isolates witness assembly from proof generation.

9.2 Prebuilt binaries and verifier path

The repository includes a path that loads prebuilt circuit binaries and a path that reconstructs circuits in memory. For deployment, the prebuilt path is preferable because it gives the runtime and the standalone verifier a stable artifact boundary. Appendices B and C record that boundary explicitly: the generated filenames, the loader paths that consume them, the `config.json` integrity check, the benchmark run identifier, and the scratch-local execution environment used for the measurements reported here.

9.3 Testing and benchmark hooks

The repository already contains integration tests for the leaf circuit, prover, verifier, and aggregation logic, as well as benchmark-specific hooks and an aggregation bench target. The timing results reported in section 10 are drawn from that benchmark surface using scratch-local runners rather than from analytical payload estimates.

10 Evaluation

We report absolute wall-clock times for the refactored aggregation implementation that is being merged into the main branch. Pre-refactor comparison numbers were collected during engineering review, but they are intentionally omitted here: the purpose of this section is to characterize the performance of the current implementation rather than to foreground deltas against an older public surface.

10.1 Methodology

Benchmarks were executed on an Apple M2 Max with 12 CPU cores and 32 GB of RAM, running macOS 26.3.1 with `rustc 1.90.0` and `cargo 1.90.0`. To reduce interference from prior builds, runs used scratch-local `HOME`, `CARGO_HOME`, `TMPDIR`, and `CARGO_TARGET_DIR` rooted under `/tmp`.

For the implemented first aggregation layer, we evaluated same-`block_hash` batches with $N \in \{2, 4, 8, 9, 16, 25, 32, 36, 49\}$ leaf proofs. For the next recursive layer, we fixed each layer-0 proof to aggregate eight leaves and varied the number of input layer-0 proofs $M \in \{2, 4, 8, 16, 32\}$.

All first-layer measurements use repeated identical dummy leaf proofs, so they remain within the implemented same-block / shared-`block_hash` protocol surface; they do not measure heterogeneous real proofs drawn from a live block. The second-layer runs likewise use repeated dummy-derived layer-0 proofs with full batches only.

Reported build times are builder wall-clock times. For layer 0, build includes leaf and layer-0 artifact generation plus configuration hashing. For layer 1, build measures layer-1 artifact generation only, with prerequisite layer-0 artifacts generated separately. Reported prove times include backend load/deserialization and the `commit()` stage. Reported verify times isolate verification from proof generation, but still include verifier load/deserialization from serialized circuit binaries.

10.2 First-layer aggregation results

Across the tested range, first-layer build times lie between 7.46 s and 30.93 s, proving times between 3.72 s and 32.42 s, and verification times between 4.50 ms and 19.58 ms. Verification remains below 12 ms for all tested sizes up to $N = 36$, with the largest tested batch ($N = 49$) verifying in 19.58 ms. These results indicate that the refactored aggregation path preserves a very small verifier footprint even when the proving side grows into the tens-of-seconds regime.

10.3 Second-layer aggregation results

For second-layer aggregation with eight leaves per input layer-0 proof, build times range from 5.54 s to 25.78 s, proving times from 3.52 s to 21.32 s, and verification times from 5.18 ms to 6.88 ms across $M \in \{2, 4, 8, 16, 32\}$. Verification is effectively flat across the tested fan-ins, while proving and artifact generation scale with the number of recursive inputs.

N leaf proofs	Build (s)	Prove (s)	Verify (ms)
2	9.20	3.83	4.50
4	7.46	3.72	4.71
8	11.30	7.32	5.53
9	11.20	7.97	5.17
16	16.69	13.47	9.26
25	16.02	16.34	11.71
32	29.45	30.84	7.51
36	28.21	32.42	7.26
49	30.93	26.66	19.58

Table 5: Measured wall-clock times for the implemented first recursive aggregation layer. All runs use same-`block_hash` dummy leaf proofs and report the current refactored implementation only.

M layer-0 proofs	Leaves per layer-0 proof	Build (s)	Prove (s)	Verify (ms)
2	8	5.56	3.52	5.18
4	8	5.54	4.23	5.34
8	8	9.22	7.21	5.94
16	8	13.27	13.86	6.39
32	8	25.78	21.32	6.88

Table 6: Measured wall-clock times for the current second-layer aggregation interface. These measurements are head-only because the explicit public layer-1 surface is introduced by the refactor.

10.4 Forwarded payload and comparison basis

The wrapper-level public-output accounting is unchanged by the refactor. For a first-layer batch of size N, the forwarded semantic payload remains $14 + 8/N$ field elements per settled leaf, excluding zero padding in the wrapper's fixed-width public-input layout. This remains the correct accounting basis for any later throughput comparison against direct post-quantum signatures.

The current timing data establish the wall-clock behavior of the refactored aggregation flow. Appendices B and C record the artifact manifest, integrity boundary, and benchmark reproduction recipe. The March 12 timing run did not persist serialized layer-0 or layer-1 proof-byte counts in the pasted CSV, so we record the exact measurement hook—serializing representative proofs with `ProofWithPublicInputs::to_bytes()` and storing the resulting lengths alongside the raw JSON outputs—rather than inventing values that were not captured.

11 Discussion and Limitations

This paper is intentionally narrower than earlier proposal prose. That narrowness is a virtue. It ensures that the text describes the implemented system rather than an aspirational superset.

Same-block scoping. The implemented first aggregation layer is same-`block_hash`. Contiguous-block aggregation is not part of the current protocol and should be understood only as historical proposal context or future work.

Two-output limit. The leaf interface supports at most two explicit exits. This is expressive enough for spend-plus-change but not for general multi-recipient batching at the leaf level.

Amount quantization. Amounts are quantized to 0.01-unit resolution in the circuit interface. This is an implementation-conscious tradeoff between circuit cost and exact arithmetic, and deployments must specify how the runtime reconstructs native precision and which edge cases this quantization excludes.

Privacy leakage. The system does not hide output amounts, exit addresses, or timing. It therefore offers a privacy boundary closer to plausible-deniability settlement than to a fully shielded ledger.

Benchmark provenance. This paper reports measured wall-clock timings for the refactored aggregation implementation and records the corresponding artifact names, integrity checks, run identifier, and raw-output naming convention in appendices B and C. A companion artifact release can extend this record with serialized proof-byte counts for the layer-0 and layer-1 benchmark outputs.

Proof granularity. The security section now states explicit main-text theorems for deposit binding, wrapper soundness, one-time withdrawal, spend-path exclusivity, and value conservation, and appendix A expands those sketches into appendix-level reductions tied to the exact verifier APIs and dummy-template invariants used in the current code paths.

12 Related Work

Wormholes and privacy pools. The conceptual starting point is EIP-7503, which proposes private proof-of-burn reminting via addresses derived from secrets [6]. Privacy Pools extends the discussion toward compliance-aware privacy sets [3]. Quantus differs by focusing on an implemented recursive proof pipeline over authenticated chain state.

Shielded and mixer-style systems. Zcash provides strong privacy through shielded notes and zero-knowledge proofs, while Tornado Cash popularized mixer-style privacy on Ethereum [5, 10]. Quantus does not attempt to duplicate the full expressiveness of shielded notes; instead it offers a lighter-weight private-settlement interface anchored directly in native chain state.

Recursive proof systems. Recursive composition is now a standard ingredient in scalable proof systems, with Halo and Plonky2 among the most visible examples [2, 9]. The Quantus contribution is not a new recursion theorem, but a concrete application of recursive composition to post-quantum-minded private settlement.

Post-quantum blockchain engineering. The adoption of ML-DSA and related schemes raises not only cryptographic questions but also execution-layer questions about transaction size, verification bottlenecks, and state growth [7]. The Quantus design explores one systems answer: compress authorization into zero-knowledge proofs rather than publishing every post-quantum authentication event directly on-chain.

13 Conclusion

Quantus Wormhole, or entangled addresses, shows how a post-quantum blockchain can move authorization off-chain without abandoning authenticated settlement. A user deposits value to

an address derived from a private secret, later proves in zero knowledge that the deposit exists in authenticated state and has not been spent before, and reveals only the settlement outputs needed by the runtime. Recursive aggregation then amortizes on-chain verification over many such withdrawals.

The resulting picture is deliberately concrete. The paper matches the implemented first-layer same-block aggregation model, includes measured timing data for the refactored aggregation flow, expands the security reductions into appendix-level arguments tied to the current dummy-template code paths, and records the artifact boundary needed for a reproducible release. Together these elements position Quantus Wormhole as a practical private-settlement architecture for post-quantum blockchains.

A Expanded proof details and code linkage

This appendix expands the main-text sketches into code-aware reductions. The goal is not to prove new claims beyond those in section 8, but to make explicit where each step relies on (i) leaf knowledge soundness, (ii) wrapper soundness, or (iii) runtime checks outside the proof system.

A.1 Dummy-template assumptions and their implementation loci

The dummy-padding argument in section 8 relies on one designated dummy template rather than on an abstract class of padding proofs. In the current implementation, that template is materialized as the serialized artifact `dummy_proof.bin`. The circuit builder writes this file in `wormhole/circuit-builder/src/lib.rs` by calling `generate_dummy_proof` from `wormhole/aggregator/src/dummy_proof.rs`. The corresponding witness is assembled by `build_dummy_circuit_inputs`, which sets `asset_id = 0`, `output_amount_1 = 0`, `output_amount_2 = 0`, `exit_account_1` $= 0^{32}$, `exit_account_2` $= 0^{32}$, `block_hash` $= 0^{32}$, and `block_number = 0`. Structurally valid placeholder values are still supplied for the storage-proof and header fields so that the witness shape matches the real leaf circuit.

The layer-0 prover can load this artifact from a binaries directory, from explicit files, or from in-memory bytes. Its `commit` path pads partial batches by cloning `dummy_proof_template`, shuffles the resulting batch while preserving a real proof in slot 0 whenever one exists, and generates one random dummy nullifier per slot. The layer-0 wrapper circuit detects dummy children with the sentinel `block_hash` $= 0^{32}$ and selects the externally supplied dummy nullifier only in those slots. This is the concrete implementation boundary behind the dummy-padding assumptions in the main text.

A.2 Detailed reduction for authenticated-deposit binding

Take any accepted *real* leaf proof (x, π) with public `block_hash` $\neq 0^{32}$. By leaf knowledge soundness, there exists an extractor that returns a witness w such that $(x, w) \in \mathcal{R}_{\text{leaf}}$. Unpacking $\mathcal{R}_{\text{leaf}}$ yields the following chain.

1. From condition (C3), the extracted witness contains a processed storage proof authenticating a transfer record under some state root rt.

2. From condition (C4), the same witness contains structured header fields whose Poseidon2 commitment equals the public `block_hash`, and whose block number equals the public `block_number`.

Claim used in the proofs	Primary file path	Concrete symbol / behavior
Designated dummy template exists as a serialized artifact	`wormhole/circuit-builder/src/lib.rs`	`generate_circuit_binaries` writes `dummy_proof.bin`.
Dummy witness uses zero block hash and zero outputs	`wormhole/aggregator/src/dummy_proof.rs`	`build_dummy_circuit_inputs`, `DUMMY_BLOCK_HASH`, `DUMMY_EXIT_ACCOUNT`.
Layer-0 padding clones exactly that template	`wormhole/aggregator/src/layer0/prover/lib.rs`	`dummy_proof_template.clone()` inside `commit`.
Real proof remains in slot 0 if any real proof exists	`wormhole/aggregator/src/layer0/prover/lib.rs`	`shuffle_proofs_preserving_first_real_layer0`.
Wrapper detects dummy children and substitutes filler nullifiers	`wormhole/aggregator/src/layer0/circuit/circuit_logic.rs`	`build_layer0_wrapper_constraints`; dummy flag from `block_hash` $= 0^{32}$.
Artifact integrity is checked against hashes stored with the run config	`wormhole/aggregator/src/config.rs`	`CircuitBinsConfig::verify_hashes`.
Layer-1 uses full batches and intentionally omits privacy padding	`wormhole/aggregator/src/aggregator.rs` `wormhole/aggregator/src/layer1/prover/lib.rs`	`Layer1Aggregator::aggregate` `Layer1AggregationProver::commit`.

Table 7: Code-level loci behind the security assumptions used in the appendix reductions.

3. Because the proof is real, it is not using the dummy sentinel. Therefore the authenticated record and the committed header are interpreted as ordinary leaf data rather than padding data.

4. The runtime checks that the public (`block_hash`, `block_number`) pair denotes a real block in canonical chain history.

5. Combining the previous two steps upgrades the self-consistent extracted header witness into a real header for a real chain block, and hence upgrades the extracted transfer record into an authenticated deposit record for that block.

6. Finally, condition (C5) yields the fee inequality for the disclosed exits.

This is the only place where the argument leaves the circuit relation itself: the leap from "self-consistent committed header" to "real chain block" is a runtime obligation, not a theorem of the leaf circuit alone.

A.3 Detailed reduction for first-layer wrapper soundness

Let Y be the public output of an accepted first-layer wrapper proof of arity N. By soundness of the recursive verifier gadgets and of the wrapper circuit, an extractor recovers the child proofs, verifier data, and dummy-nullifier witness values used by the wrapper. The reduction then proceeds in four stages.

1. The wrapper circuit allocates N child proof targets and verifies each one under the same leaf verifier data. Therefore each extracted child proof is itself an accepted leaf proof whenever its proof target is populated by a real child witness.

2. In `build_layer0_wrapper_constraints`, the wrapper defines a dummy flag δ_i by testing whether child i has public `block_hash` equal to 0^{32}. For every child i, the circuit enforces

$$\delta_i \vee (\texttt{block_hash}_i = \texttt{block_hash}_{\text{ref}}),$$

and always enforces

$$\texttt{asset_id}_i = \texttt{asset_id}_{\text{ref}}, \qquad \texttt{volume_fee_bps}_i = \texttt{volume_fee_bps}_{\text{ref}}.$$

Here the reference values are taken from slot 0.

3. The layer-0 prover's `commit` path shuffles the batch with `shuffle_proofs_preserving_first_real_layer0`. Hence, if there exists any real child, slot 0 is guaranteed to hold a real child. If all children are dummy, the wrapper's header output is the all-zero dummy header, which is harmless because the runtime settles only the forwarded exits.

4. The wrapper's exit logic iterates over the $2N$ child exit positions. For each output slot it treats that slot's exit account as a key, sums all matching child amounts, and zeroes the slot if the same exit key already appeared earlier. This is exactly a deterministic partition of the multiset of child exits by destination account, followed by one representative slot per equivalence class.

5. The wrapper's nullifier logic outputs the real child nullifier when $\delta_i = 0$ and the externally supplied dummy filler when $\delta_i = 1$.

One subtlety is the `block_number` field. The wrapper circuit outputs the reference child's public `block_number`, but it does not separately `connect` every other child's `block_number` target to that value. The paper's common-`block_number` claim therefore comes from combining wrapper soundness with leaf soundness: every real child is itself a sound leaf proof satisfying condition (C4), and every real child shares the same public `block_hash`. Because the leaf relation binds `block_number` into the same committed header preimage, a common `block_hash` across sound leaves induces a common `block_number` as well.

A.4 Detailed reduction for dummy-padding safety

The dummy-padding safety lemma in the main text is a composition of three code-level facts.

1. The designated dummy template produced by `build_dummy_circuit_inputs` has zero-valued public outputs and zero exit accounts. Therefore every padded child contributes the all-zero exit tuple before wrapper grouping.

2. The layer-0 prover pads only by cloning this designated template. It does not synthesize any alternative dummy object. When real proofs are mixed with dummies, the prover also checks that the real proofs use the same zero asset identifier as the dummy template, matching the wrapper's across-batch equality constraint.

3. The wrapper groups exits by account and zeros duplicates. Since every dummy child contributes zero amounts to the all-zero exit account, grouping or deduplicating dummy exits cannot increase the sum of any forwarded exit amount.

The only remaining effect of dummies is on the nullifier region. There the wrapper uses a fresh externally supplied filler for each dummy slot. This substitution is value-neutral: it affects only whether a candidate aggregate might be rejected because a filler accidentally collides with an already-used nullifier. That is a liveness concern, not a soundness concern.

A.5 Detailed reduction for one-time withdrawal

Suppose, for contradiction, that an adversary obtains two accepted *real* withdrawals from the same authenticated deposit record d. By authenticated-deposit binding, extraction yields witnesses (s, c) and (s', c') together with the same authenticated record

$$d = (a, v_{\text{in}}, c, \alpha_{\text{fund}}, \alpha_{\text{wh}}, h_B, b).$$

Because the authenticated record fixes the transfer counter, we have $c = c'$. Because the same record also fixes the entangled address, both extracted witnesses satisfy

$$\mathsf{EA}(s) = \alpha_{\text{wh}} = \mathsf{EA}(s').$$

If $s \neq s'$, the adversary has found a collision or second preimage for the entangled-address derivation. Therefore $s = s'$ except with negligible probability. Substituting back into the nullifier derivation gives

$$\mathsf{Null}(s, c) = \mathsf{Null}(s', c').$$

So the two accepted withdrawals expose the same public nullifier. The runtime rejects the second occurrence by the nullifier-freshness rule. Hence one authenticated deposit record cannot underwrite two accepted real withdrawals except with negligible probability.

A.6 Detailed reduction for global value conservation

Take any accepted first-layer aggregate.

1. By wrapper soundness, the aggregate expands into a list of real children and dummy padding.
2. By dummy-padding safety, dummy children contribute no settled value.
3. By authenticated-deposit binding, every real child corresponds to an authenticated deposit record.
4. By the per-leaf fee bound, each real child's two exits sum to at most the net deposit amount authorized by its authenticated record.
5. By wrapper amount preservation, grouping identical exit accounts preserves the total sum of the real child exits.
6. By one-time withdrawal per authenticated deposit record and by runtime nullifier freshness, no authenticated record can be counted twice across accepted aggregates.

Therefore the total value settled by any collection of accepted first-layer aggregates is upper bounded by the total net value of the distinct authenticated deposit records that underlie their real children.

B Artifact manifest and integrity boundary

The implementation uses a prebuilt-artifact boundary rather than reconstructing circuits in memory at every runtime entry point. This appendix records the concrete filenames and the loader paths that consume them so that the eventual artifact bundle can be audited against the code.

The integrity boundary is encoded in `config.json`. Besides the batch-size parameters for the layer-0 and layer-1 fan-in, the config stores SHA-256 hashes for every artifact present in the directory. Both `Layer0Aggregator::new` and `Layer1Aggregator::new` load this file and verify those hashes before accepting the binaries as trustworthy inputs. This prevents a mixed-artifact configuration in which, for example, a layer-0 prover is loaded against stale or mismatched common data.

B.1 Manifest fields for an artifact release

A release-quality artifact bundle should ship a machine-readable manifest alongside the binaries. At minimum it should include:

1. repository commit hash and branch name;
2. benchmark run identifier;
3. host information (CPU model, RAM, OS version, Rust toolchain);
4. the full batch-shape parameters (N, M) used for each reported run;
5. one record per artifact file containing filename, SHA-256 hash, and byte length; and
6. one record per representative proof containing proof family (*leaf*, *layer-0*, or *layer-1*), batch shape, serialized byte length, and public-input length.

B.2 Proof-size measurement boundary

Proof sizes should be measured from the exact serialized proof objects that are already produced during benchmarking, using the same serialization boundary that the runtime consumes, namely `ProofWithPublicInputs::to_bytes()`. The leaf verifier benchmark already demonstrates this boundary through the committed `wormhole/bench-data/proof.bin` artifact. The March 12 timing campaign whose wall-clock results appear in section 10 did not persist serialized layer-0 or layer-1 proof-byte counts in the pasted CSV, so we record the measurement procedure and required manifest fields rather than reporting unobserved numbers.

C Benchmark reproducibility details

The benchmark tables in section 10 are derived from a scratch-local campaign identified by the run directory

`/tmp/quantus-bench-20260312-133708`.

The reported numbers intentionally use only the `head_ms` columns of that campaign, because the paper characterizes the refactored implementation itself rather than foregrounding a pre-refactor delta.

The raw-result filenames should also be preserved because they encode the batch shape. The head-only timing files follow the conventions:

```
results/raw/head/layer0-build-n<N>.json
results/raw/head/layer0-prove-verify-n<N>.json
results/raw/head/layer1-build-l0leaves8-n<M>.json
results/raw/head/layer1-prove-verify-l0leaves8-n<M>.json
```

A release-quality artifact bundle should include these raw JSON files verbatim together with the normalized CSV used to typeset the tables.

At the command level, circuit artifacts are generated by the builder CLI. For layer 0 only, the relevant invocation is

```
cargo run -release -p
qp-wormhole-circuit-builder - -output wormhole/generated-bins
-num-leaf-proofs <N>
```

and for runs that also build the explicit layer-1 surface,

```
cargo run -release -p
qp-wormhole-circuit-builder - -output wormhole/generated-bins
-num-leaf-proofs 8 -num-layer0-proofs <M>
```

The repository also includes Criterion entry points for leaf proving, leaf verification, and aggregation under the crates `qp-wormhole-prover`, `qp-wormhole-verifier`, and `qp-wormhole-aggregator`. The March 12 campaign used custom scratch harnesses rather than only the stock Criterion macros, which is why the paper records the output schema and environment explicitly here.

D Privacy-score heuristic

This appendix records a user-facing privacy-score heuristic intended to estimate how many deposit subsets might plausibly explain an observed withdrawal. It is a wallet-side heuristic and should not be interpreted as a formal privacy guarantee.

Let $\mathcal{D}$ denote the candidate deposit multiset, let V denote the observed effective withdrawal amount after fees, and let f denote the fee basis points. A heuristic score can be defined as the base-2 logarithm of the number of subsets whose fee-adjusted sum falls into a tolerance window around V:

$$\mathsf{Score}(V, \mathcal{D}) = \log_2 \left| \left\{ S \subseteq \mathcal{D} : \sum_{d \in S} d \cdot (1 - f) \approx V \right\} \right|.$$

A central-limit-style approximation over deposit buckets may be sufficient for wallet guidance, but this score remains a heuristic rather than a formal privacy guarantee.

E Exact public-input tables

For implementation-facing completeness, we preserve the exact leaf and wrapper layouts.

References

[1] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast Reed-Solomon Interactive Oracle Proofs of Proximity. In *45th International Colloquium on Automata, Languages, and Programming (ICALP)*, 2018.

[2] Sean Bowe, Jack Grigg, and Daira Hopwood. Recursive Proof Composition without a Trusted Setup. Cryptology ePrint Archive, Report 2019/1021, 2019.

[3] Vitalik Buterin, Jacob Illum, Matthias Nadler, Fabian Sch"ar, and Ameen Soleimani. Blockchain Privacy and Regulatory Compliance: Towards a Practical Equilibrium. SSRN, 2023.

[4] Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. Poseidon2: A Faster Version of the Poseidon Hash Function. Cryptology ePrint Archive, Report 2023/323, 2023.

[5] Daira Hopwood, Sean Bowe, Taylor Hornby, and Nathan Wilcox. Zcash Protocol Specification. Protocol specification, 2023.

[6] Keyvan Kambakhsh, Hamid Bateni, Amir Kahoori, and Nobitex Labs. EIP-7503: Zero-Knowledge Wormholes. Ethereum Improvement Proposal, 2023.

[7] National Institute of Standards and Technology. Module-Lattice-Based Digital Signature Standard (FIPS 204). Federal Information Processing Standards Publication 204, 2024.

[8] Parity Technologies. Substrate: The Blockchain Framework for a Multichain Future. Project documentation, 2023.

[9] Polygon Zero Team. Plonky2: Fast Recursive Arguments with PLONK and FRI. Software repository, 2022.

[10] Alexey Pertsev, Roman Semenov, and Roman Storm. Tornado Cash Privacy Solution. Project documentation, 2019.

[11] Quantus Network. qp-zk-circuits. Software repository, 2026.

[12] Peter W. Shor. Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.

[13] Christopher Smith. QIP-0005: Entangled Addresses: Scalable Post-Quantum ZK-Signatures. Quantus Improvement Proposal draft, 2025.

Artifact	Produced by	Consumed by
`common.bin`	`generate_circuit_binaries`	leaf verifier benchmark, leaf prover / verifier loaders, layer-0 builder
`verifier.bin`	`generate_circuit_binaries`	leaf verifier loader, layer-0 prover loader
`prover.bin`	`generate_circuit_binaries`	leaf prover loader
`dummy_proof.bin`	`generate_dummy_proof` via `generate_circuit_binaries`	layer-0 prover loader; batch padding template
`aggregated_common.bin`	`generate_layer0_circuit_binaries`	layer-0 verifier loader, layer-1 builder, layer-1 prover loader
`aggregated_verifier.bin`	`generate_layer0_circuit_binaries`	layer-0 verifier loader, layer-1 prover loader
`aggregated_prover.bin`	`generate_layer0_circuit_binaries`	layer-0 prover loader
`layer1_common.bin`	`generate_layer1_circuit_binaries`	layer-1 verifier loader, layer-1 prover loader
`layer1_verifier.bin`	`generate_layer1_circuit_binaries`	layer-1 verifier loader
`layer1_prover.bin`	`generate_layer1_circuit_binaries`	layer-1 prover loader
`config.json`	`CircuitBinsConfig::save`	layer-0 and layer-1 loaders; hash verification and batch-size metadata

Table 8: Concrete artifact names used by the refactored prover / verifier / aggregator stack.

Field	Value
Implementation under test	refactor branch head selected for the benchmark campaign (`f52e5c917352a0a056c78e7ab39b94b0a367b40c`)
Machine	Apple M2 Max, 12 CPU cores, 32 GB RAM
OS	macOS 26.3.1
Toolchain	`rustc 1.90.0`, `cargo 1.90.0`
Scratch execution root	`/tmp/quantus-bench-20260312-133708`
Scratch-local environment variables	`HOME`, `CARGO_HOME`, `TMPDIR`, and `CARGO_TARGET_DIR` were rooted under the scratch directory; `RUSTUP_HOME` remained host-global
Layer-0 matrix	$N \in \{2, 4, 8, 9, 16, 25, 32, 36, 49\}$ leaf proofs
Layer-1 matrix	eight leaves per layer-0 proof and $M \in \{2, 4, 8, 16, 32\}$ input layer-0 proofs
Workload model	repeated identical dummy leaf proofs for layer 0; repeated dummy-derived layer-0 proofs for layer 1
Comparison policy	report `head_ms` only; ignore baseline and ratio fields in the paper narrative

Table 9: Benchmark provenance for the timing tables reported in section 10.

Index range	Field	Size (felts)
0	`asset_id`	1
1	`output_amount_1`	1
2	`output_amount_2`	1
3	`volume_fee_bps`	1
4–7	`nullifier`	4
8–11	`exit_account_1`	4
12–15	`exit_account_2`	4
16–19	`block_hash`	4
20	`block_number`	1

Table 10: Leaf public-input layout (21 felts total).

Offset	Field	Size
0	`num_exit_slots`	1
1	`asset_id`	1
2	`volume_fee_bps`	1
3–6	`block_hash`	4
7	`block_number`	1
8..	`[sum, exit_account]` repeated $2N$ times	5 per slot
next	`nullifier` repeated N times	4 per nullifier
rest	zero padding	until $21N + 8$ felts

Table 11: First-layer aggregate public-input layout.